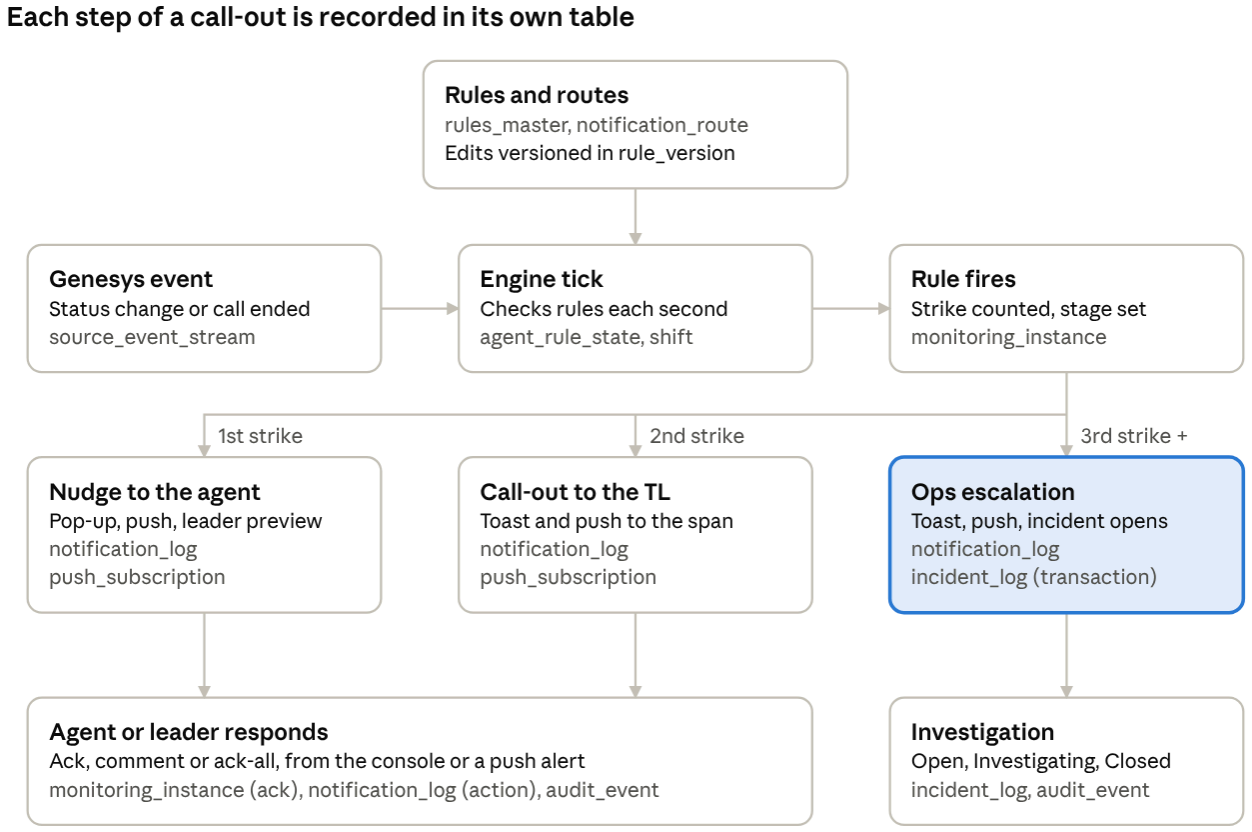
RTM Console PostgreSQL Schema

Oct 12, 2026 · @Adrian

The console needs 20 PostgreSQL tables: the six from the *RTM Dashboard Data Tables* file, two child tables that split out its route map and rule history, eight that cover people, shift state, delivery and audit, and four for Genesys regions, orgs, queues and queue membership. Business rules are enforced by keys, checks and one partial unique index, not by application code. This replaces the MongoDB version of this doc.

Process flow



call-out flow · 8 steps, 3 escalation stages

The ladder shown is the `nudge` route; the `lead` route starts at the TL, and the two capped routes stop at their first rung. The highlighted step is the only one that must write several tables in one transaction.

Tables

`monitoring_instance` and `notification_log` are the high-volume business records; `source_event_stream` is the largest by far. The rest is configuration, reference data or recovery state.

Table	From the Word file	Holds	Rows	Written today by
`source_event_stream`	SourceEventStream	Raw events from Genesys, NICE or UiPath, partitioned by month	Largest: tens of thousands per shift	`onAgentState`, `onQueue`
`rules_master`	RulesMaster	One row per rule: name, type, scope, metric, operator, current version	14	`defaultRules()`
`rule_version`	Split from RulesMaster	Every version of a rule's threshold, unit, severity, route and on/off	Low	`setThr`, `setSev`, `setRoute`, `setOn`
`notification_route`	NotificationRoute	The four escalation routes	4	`ROUTES` in `lib/engine/rules.ts`
`notification_route_step`	RouteMap, as rows	Each rung: occurrences, stage, recipients, action owner, action, channels	About 8	`stageFor()`, `alertsFor()`
`monitoring_instance`	MonitoringInstance	One row per call-out, with ack and the agent's comment	Hundreds to thousands per shift	`fire()`, `ack`, `ackAll`, `comment`
`incident_log`	IncidentLog	Investigations opened at strike 3	Tens per shift	`openIncident()`, `invAction`
`notification_log`	NotificationLog	One row per recipient per channel, with delivery status and action	Several per call-out	Toast and nudge hooks, `pushBatch()`
`push_subscription`	New	Browser push endpoints per person	One per browser	`saveSubscription` in `lib/server/push.ts`
`person`	New	Agents and leaders by Genesys user ID, with role, account state and last login	Hundreds	onRoster (from queue members), later SSO
`team`	New	Team, LOB, sub-LOB, TL and manager	Tens	`onRoster`
`shift`	New	Shift boundaries, data source, floor rules in breach	One per shift	`engine.reset()`
`agent_rule_state`	New	Strike count and fired flag per agent, rule and shift	Agents × rules	`agent.strikes`, `agent.fired`

Table	From the Word file	Holds	Rows	Written today by
`incident_counter`	New	Last incident number per year	One per year	`S.incSeq`
`status_mapping`	New	Ordered rules turning Genesys presence, routing and after-call work into a console state	Tens	`mapGenesysState()` and `consoleState()` in `lib/feed/gencloud/state.ts`; `guessState()` for CSV replays
`audit_event`	New	Incident status changes, ack-all, feed switches, token refreshes	Low	Incident, ack-all, feed and token routes
`genesys_region`	New	Genesys regions and their API and login hosts	2 now (US East, US West)	Seeded; not yet read
`genesys_org`	New	Each watched org: region, watched view, OAuth client	Several per region	Admin setup; today one org from .env
`queue`	New	Each Genesys queue: org, division, queue group, whether the org's view watches it, its team	1,378 readable in anthem-commercial; 90 watched	`getWatchedQueues()`, `tools/list-queues.mjs`
`queue_member`	New	One person on one queue	44,386 in the watched view	`getQueueMembers()`

Every table hangs off monitoring_instance

Fields with → are foreign keys; the arrow runs to the table they name. core = one of the six tables in the Word file





table map · 20 tables, 7 groups

`monitoring_instance` sits at the centre: incidents, notifications and strike state all point back to it or to the people it names. Dark field lines are foreign keys; boxes tagged *core* are the six tables from the Word file.

Rules and routes

A rule's editable settings live in `rule_version`, so every edit keeps the old values and each call-out points at the exact version that fired it. The Word file's `RouteMap` JSON becomes one row per rung, which lets the database check every step.

```
CREATE TABLE notification_route (
  route_id     text PRIMARY KEY,              -- 'nudge', 'lead',
'nudgeonly', 'leadonly'
  route_name   text NOT NULL,
  description  text,
  enabled      boolean NOT NULL DEFAULT true
);

CREATE TABLE notification_route_step (
  route_id        text NOT NULL REFERENCES notification_route,
  step_no         smallint NOT NULL,
  min_occurrence  smallint NOT NULL,          -- 0 = floor call-outs
  max_occurrence  smallint,                   -- NULL = this and every later
occurrence
  stage           text NOT NULL CHECK (stage IN ('nudge', 'lead', 'ops')),
  recipient_roles text[] NOT NULL,            -- {agent} or
{tl,mgr,admin,senior}
  action_owner    text NOT NULL,              -- 'agent', 'tl', 'mgr'
  action          text NOT NULL,              -- 'Review', 'Coaching',
'Create incident'
  channels        text[] NOT NULL,            -- {popup,push} or {toast,push}
  opens_incident  boolean NOT NULL DEFAULT false,
  PRIMARY KEY (route_id, step_no),
  CHECK (max_occurrence IS NULL OR max_occurrence >= min_occurrence)
);

CREATE TABLE rules_master (
  rule_id          text PRIMARY KEY,          -- 'acw', 'auxp', ... (14
```

```
rules)
  rule_name        text NOT NULL,
  description      text,
  rule_type        text NOT NULL CHECK (rule_type IN ('duration', 'event',
'ratio', 'queue', 'system')),
  rule_scope       text NOT NULL CHECK (rule_scope IN ('agent', 'floor')),
  metric_name      text NOT NULL,            -- 'ACW duration', 'Service
level'
  operator         text NOT NULL CHECK (operator IN ('>', '>=', '<', '<=')),
  once_per_shift   boolean NOT NULL DEFAULT false,
  nudge_template   text,
  current_version  integer NOT NULL,
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE rule_version (
  rule_id     text NOT NULL REFERENCES rules_master,
  version     integer NOT NULL,
  threshold   numeric NOT NULL CHECK (threshold >= 1),
  unit        text NOT NULL,                 -- 's', '%', 'calls'
  severity    text NOT NULL CHECK (severity IN ('warn', 'crit')),
  route_id    text NOT NULL REFERENCES notification_route,
  enabled     boolean NOT NULL,
  changed_by  text REFERENCES person,
  changed_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (rule_id, version)
);

ALTER TABLE rules_master
  ADD FOREIGN KEY (rule_id, current_version) REFERENCES rule_version
  DEFERRABLE INITIALLY DEFERRED;
```

- A rule edit inserts a new `rule_version` row and bumps `rules_master.current_version` in one transaction. The current settings are a join on `current_version`.
- The `nudge` route holds three steps, matching today's ladder: occurrences 0 to 1 nudge the agent and preview to leaders; 2 goes to the TL; 3 and later go to Ops with `opens_incident = true`.
- `rule_scope` allows only `agent` and `floor`, the two levels the engine evaluates. Widen the check when team or LOB rules exist.

Instances and incidents

Every breach is its own `monitoring_instance` row, because the ledger, acknowledgements, comments and dashboard counts all work per call-out. An incident gathers many instances through `monitoring_instance.incident_id`, and a partial unique index allows only one open incident per agent per rule.

```
CREATE TABLE incident_counter (
  year        smallint PRIMARY KEY,
  last_value  integer NOT NULL DEFAULT 0
);

CREATE TABLE incident_log (
  incident_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  incident_number  text NOT NULL UNIQUE,      -- 'INC-2026-0001'
  shift_id         text NOT NULL REFERENCES shift,
  source           text NOT NULL CHECK (source IN ('gencloud', 'sim',
'csv')),
  person_id        text NOT NULL REFERENCES person,
  team_id          text NOT NULL REFERENCES team,   -- team when it opened
  rule_id          text NOT NULL REFERENCES rules_master,
  status           text NOT NULL DEFAULT 'Open' CHECK (status IN ('Open',
'Investigating', 'Closed')),
  disposition      text CHECK (disposition IN ('Coached & documented',
'Validated / excused',
                                               'Referred to HR / PIP',
'System issue / no fault')),
  owner_id         text REFERENCES person,
  priority         text,
  root_cause       text,
  resolution       text,
  comments         text,
  opened_at        timestamptz NOT NULL DEFAULT now(),
  closed_at        timestamptz,
  modified_at      timestamptz NOT NULL DEFAULT now(),
  CHECK ((status = 'Closed') = (closed_at IS NOT NULL AND disposition IS NOT
NULL))
);

CREATE UNIQUE INDEX one_open_incident
  ON incident_log (person_id, rule_id) WHERE status <> 'Closed';

CREATE TABLE monitoring_instance (
  instance_id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shift_id            text NOT NULL REFERENCES shift,
  source              text NOT NULL CHECK (source IN ('gencloud', 'sim',
```

```
'csv')),
  rule_id             text NOT NULL,
  rule_version        integer NOT NULL,
  person_id           text REFERENCES person,      -- NULL for queue and
system rules
  team_id             text REFERENCES team,        -- team at fire time
  metric_value        text NOT NULL,               -- as shown: '2:00',
'72%', '25% (5/20)'
  threshold_value     numeric NOT NULL,            -- copied from
rule_version for reports
  condition_text      text NOT NULL,               -- 'State ACW longer than
120s'
  severity            text NOT NULL CHECK (severity IN ('warn', 'crit',
'esc')),
  occurrence_count    smallint NOT NULL,           -- this breach's strike; 0
for floor call-outs
  notification_level  text NOT NULL CHECK (notification_level IN ('nudge',
'lead', 'ops')),
  status              text NOT NULL DEFAULT 'open' CHECK (status IN ('open',
'acked')),
  acknowledged_by     text REFERENCES person,
  acknowledged_at     timestamptz,
  ack_via             text CHECK (ack_via IN ('console', 'push', 'ack_all')),
  agent_comment       text CHECK (char_length(agent_comment) <= 140),
  agent_comment_at    timestamptz,
  incident_id         bigint REFERENCES incident_log,
  source_event_id     bigint,                      -- no FK: see below
  created_at          timestamptz NOT NULL DEFAULT now(),
  updated_at          timestamptz NOT NULL DEFAULT now(),
  FOREIGN KEY (rule_id, rule_version) REFERENCES rule_version,
  CHECK ((person_id IS NULL) = (occurrence_count = 0)),
  CHECK ((status = 'acked') = (acknowledged_at IS NOT NULL))
);
```

- An incident's instance count is `count(*)` of the instances pointing at it, so it can never drift.
- `source_event_id` has no foreign key. Duration rules fire from a timer, not from one event, so it is often NULL; and `source_event_stream` is partitioned and pruned, which a foreign key would block.
- `incident_number` comes from `incident_counter` inside the strike-3 transaction (see Indexes and key queries).

Notifications

`notification_log` keeps one row per recipient per channel, so it can record that one leader's push failed while the others went through, and what each person did with it. As in the Word file, it stores tracking only, not the message text.

```
CREATE TABLE push_subscription (
  endpoint_hash    text PRIMARY KEY,           -- sha256 of the endpoint
  endpoint         text NOT NULL,
  p256dh           text NOT NULL,
  auth             text NOT NULL,
  person_id        text NOT NULL REFERENCES person,
  user_agent       text,
  created_at       timestamptz NOT NULL DEFAULT now(),
  last_success_at  timestamptz
);

CREATE TABLE notification_log (
  notification_log_id  bigint GENERATED ALWAYS AS IDENTITY,
  notification_ts      timestamptz NOT NULL DEFAULT now(),
  instance_id          bigint REFERENCES monitoring_instance,   -- NULL for
feed notices
  recipient_id         text NOT NULL REFERENCES person,
  recipient_role       text NOT NULL,
  channel              text NOT NULL CHECK (channel IN ('popup', 'toast',
'preview', 'push', 'email', 'teams')),
  notification_level   text NOT NULL CHECK (notification_level IN ('nudge',
'lead', 'ops', 'info')),
  endpoint_hash        text REFERENCES push_subscription ON DELETE SET NULL,
  delivery_status      text NOT NULL CHECK (delivery_status IN ('sent',
'shown', 'delivered', 'expired', 'gone', 'failed')),
  action               text CHECK (action IN ('got_it', 'send_reason',
'acknowledge')),
  action_at            timestamptz,
  PRIMARY KEY (notification_log_id, notification_ts)
) PARTITION BY RANGE (notification_ts);
```

- Partitioned by month, so retention is dropping an old partition rather than a large `DELETE`.
- `gone` means the push service answered 404 or 410; the app then deletes the `push_subscription` row, which sets `endpoint_hash` to NULL on past log rows.
- `email` and `teams` are allowed for the channels the Word file plans; the app does not send them yet.

Events, people and shift state

These tables give role scoping something to join on, let the engine resume mid-shift after a restart, and keep raw events for replay and disputes.

```
CREATE TABLE person (
  person_id      text PRIMARY KEY,             -- Genesys user ID
  name           text NOT NULL,
  email          text,
  role           text NOT NULL CHECK (role IN ('admin', 'senior', 'mgr',
'tl', 'agent')),
  team_id        text,                         -- FK added after team exists
  account_state  text NOT NULL DEFAULT 'active' CHECK (account_state IN
('active', 'inactive', 'deleted')),
  last_login_at  timestamptz                   -- Genesys dateLastLogin
);

CREATE TABLE team (
  team_id     text PRIMARY KEY,
  name        text NOT NULL,
  lob         text NOT NULL,
  sub_lob     text,
  tl_id       text REFERENCES person,
  manager_id  text REFERENCES person
);

ALTER TABLE person ADD FOREIGN KEY (team_id) REFERENCES team;

CREATE TABLE shift (
  shift_id     text PRIMARY KEY,               -- '2026-10-07-day'
  starts_at    timestamptz NOT NULL,
  ends_at      timestamptz NOT NULL,
  source       text NOT NULL CHECK (source IN ('gencloud', 'sim')),
  status       text NOT NULL CHECK (status IN ('active', 'closed')),
  floor_fired  text[] NOT NULL DEFAULT '{}'    -- floor rules in breach now,
e.g. {sl}
);

CREATE UNIQUE INDEX one_active_shift_per_source ON shift (source) WHERE
status = 'active';

CREATE TABLE agent_rule_state (
  shift_id   text NOT NULL REFERENCES shift,
  person_id  text NOT NULL REFERENCES person,
  rule_id    text NOT NULL REFERENCES rules_master,
  strikes    smallint NOT NULL DEFAULT 0,
```

```
  fired      boolean NOT NULL DEFAULT false,   -- already fired in the
current state episode
  PRIMARY KEY (shift_id, person_id, rule_id)
);

CREATE TABLE source_event_stream (
  event_id        bigint GENERATED ALWAYS AS IDENTITY,
  event_ts        timestamptz NOT NULL,
  source_system   text NOT NULL,               -- 'GenCloud', 'NICE',
'UiPath'
  entity_type     text NOT NULL CHECK (entity_type IN ('agent', 'queue',
'team', 'lob', 'system')),
  entity_id       text NOT NULL,
  metric_name     text NOT NULL,               -- 'presence',
'routingStatus', 'call.end', 'hold', 'acw', 'oWaiting', 'tHandle'
  metric_value    text,
  event_payload   jsonb,                       -- the original payload
  processed_flag  boolean NOT NULL DEFAULT false,
  processed_at    timestamptz,
  PRIMARY KEY (event_id, event_ts)
) PARTITION BY RANGE (event_ts);

-- Checked in priority order; the first row that matches wins. No match =
'auxp'.
CREATE TABLE status_mapping (
  priority        smallint PRIMARY KEY,
  signal          text NOT NULL CHECK (signal IN ('presence', 'routing',
'call')),
  genesys_status  text NOT NULL,               -- 'OFFLINE', 'INTERACTING',
'ACW_NO_CALL'
  console_state   text NOT NULL CHECK (console_state IN ('oncall', 'avail',
'acw', 'auxb', 'auxp', 'outb', 'off')),
  updated_by      text REFERENCES person,
  updated_at      timestamptz NOT NULL DEFAULT now(),
  UNIQUE (signal, genesys_status)
);

INSERT INTO status_mapping (priority, signal, genesys_status, console_state)
VALUES
  (10, 'presence', 'OFFLINE',        'off'),
  (20, 'call',     'ACW_NO_CALL',    'acw'),   -- after-call work pending, no
call connected
  (30, 'routing',  'INTERACTING',    'oncall'),
  (40, 'routing',  'COMMUNICATING',  'outb'),
  (50, 'routing',  'IDLE',           'avail'),
  (60, 'routing',  'NOT_RESPONDING', 'auxp'),
```

```
  (70, 'presence', 'BREAK',          'auxb'),
  (80, 'presence', 'MEAL',           'auxb'),
  (90, 'presence', 'ON_QUEUE',       'avail');

CREATE TABLE audit_event (
  audit_id     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  at           timestamptz NOT NULL DEFAULT now(),
  actor_id     text REFERENCES person,
  action       text NOT NULL,                  -- 'incident.start',
'instance.ack_all', 'feed.switch', 'token.refresh'
  target_type  text NOT NULL,
  target_id    text,
  before       jsonb,
  after        jsonb
);
```

- Agent timers (`stTime` , `holdTime`) are not stored. After a restart the engine reloads `agent_rule_state` and takes a fresh status snapshot from Genesys. Its presence modifiedDate and routing startTime give each agent's time already in state (counted from Eastern midnight at most, except calls), and rules already past are marked fired without a call-out.
- Queue metrics are `source_event_stream` rows with `entity_type = 'queue'` , as in the Word file's "LOB Service Level" example, so they need no table of their own.
- LOB and sub-LOB live on `team` only. Instances and incidents carry `team_id` and join for them, so a correction is made in one place.
- `status_mapping` is a list checked top down, because one Genesys status cannot decide the state alone: Offline beats everything, pending after-call work beats routing (which stays INTERACTING through ACW), and routing beats presence. The seed rows match `mapGenesysState()` and `consoleState()` today.
- Call events (call end, transfer, hold, after-call work) land in `source_event_stream` with `entity_type = 'agent'` ; calls handled, transfers and short calls can be rebuilt from them after a restart.

Regions, organizations, queue groups and queues

Two tables record where each Genesys org lives, and an `org_id` column ties people, teams and events to their org. Each region holds several orgs; each org has its own token, users and queues.

```
CREATE TABLE genesys_region (
  region_id   text PRIMARY KEY,                -- 'us-east-1', 'us-west-2'
  name        text NOT NULL,                   -- 'US East'
  api_host    text NOT NULL,                   -- 'api.mypurecloud.com'
  login_host  text NOT NULL                    -- 'login.mypurecloud.com'
);

INSERT INTO genesys_region VALUES
  ('us-east-1', 'US East', 'api.mypurecloud.com', 'login.mypurecloud.com'),
  ('us-west-2', 'US West', 'api.usw2.pure.cloud', 'login.usw2.pure.cloud');

CREATE TABLE genesys_org (
  org_id           text PRIMARY KEY,           -- Genesys organization ID
  region_id        text NOT NULL REFERENCES genesys_region,
  name             text NOT NULL,              -- 'anthem-commercial'
  view_config_id   text,                       -- the watched queues
  oauth_client_id  text,                       -- the secret lives in the
secret store, never here
  enabled          boolean NOT NULL DEFAULT true
);

ALTER TABLE person ADD COLUMN org_id text NOT NULL REFERENCES genesys_org;
ALTER TABLE team   ADD COLUMN org_id text NOT NULL REFERENCES genesys_org;
ALTER TABLE source_event_stream ADD COLUMN org_id text REFERENCES
genesys_org;  -- null for NICE and UiPath
```

Queues sit inside an org. Today each watched queue is also the console's team, and an agent belongs to many queues, so membership needs its own table.

```
CREATE TABLE queue (
  queue_id        text PRIMARY KEY,            -- Genesys queue ID
  org_id          text NOT NULL REFERENCES genesys_org,
  name            text NOT NULL,               --
'CSBD_LGO_LOCAL_PROV_CENTRAL_WGS_CLAIMS'
  division        text,                        -- 'CSBD_PROV_PRD'
  queue_group     text GENERATED ALWAYS AS (split_part(name, '_', 2)) STORED,
-- 'LGO'
  queue_subgroup  text GENERATED ALWAYS AS (split_part(name, '_', 3)) STORED,
-- 'LOCAL'
  watched         boolean NOT NULL DEFAULT false,  -- in the org's
view_config_id
  team_id         text REFERENCES team,        -- today: a team of the same
name
  seen_at         timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE queue_member (
  queue_id   text NOT NULL REFERENCES queue ON DELETE CASCADE,
  person_id  text NOT NULL REFERENCES person,
  PRIMARY KEY (queue_id, person_id)
);

-- The live roster: active accounts on a watched queue with a login in the
last 30 days
SELECT DISTINCT p.*
FROM person p
JOIN queue_member m ON m.person_id = p.person_id
JOIN queue q ON q.queue_id = m.queue_id AND q.watched
WHERE p.account_state = 'active'
  AND (p.last_login_at IS NULL OR p.last_login_at >= now() - interval '30
days');
```

- **One person, one org** is assumed and still to be confirmed. If a person can work in several orgs, add `genesys_account (org_id, user_id)` pointing at `person`.
- **Deactivated accounts** load with `person.account_state = 'inactive'` and stay off the floor. The watched view had 1,039 on 9 Oct 2026, and the roster also leaves out the 116 active accounts with no login in 30 days.
- **Dates are Eastern** (America/New_York): `shift_id` dates and the queue numbers' "today" start at Eastern midnight.
- **Queue groups** are read from the name (`CSBD_<group>_<subgroup>_…`): LGO 51, ELV 22, CGS 9 and CNX 8 of the 90 watched queues. The generated columns hold only while queues keep that naming; if groups need owners, thresholds or names of their own, promote them to a `queue_group` table.
- **Watched** comes from the org's saved view and is refreshed at each connect; the view is edited in Genesys, not here.

Indexes and key queries

Each index serves a query the app already makes; the two partial unique indexes enforce business rules.

Table	Index	Serves
`monitoring_instance`	`(shift_id, team_id, created_at DESC)`	Ledger and stream snapshot for a span
`monitoring_instance`	`(shift_id, person_id, rule_id)`	Rebuilding strikes, the agent's own view

Table	Index	Serves
`monitoring_instance`	`(person_id) WHERE status = 'open' AND notification_level = 'nudge'`	`pendingNudge()`
`monitoring_instance`	`(incident_id)`	An incident's call-outs and count
`incident_log`	`UNIQUE (person_id, rule_id) WHERE status <> 'Closed'`	One open incident per agent per rule
`incident_log`	`(team_id, status)`	Incident tiles and register
`team`	`(tl_id)`, `(manager_id)`	Role scoping
`notification_log`	`(instance_id)`, `(recipient_id, notification_ts DESC)`	Delivery trail per call-out and per person
`push_subscription`	`(person_id)`	Fan-out in `pushBatch()`
`source_event_stream`	`(entity_id, event_ts)`, `(event_ts) WHERE NOT processed_flag`	Replay, disputes, the UiPath queue
`shift`	`UNIQUE (source) WHERE status = 'active'`	One active shift per data source
`queue_member`	`(person_id)`	An agent's queues, the roster join
`queue`	`(org_id) WHERE watched`, `(queue_group)`	The watched queue list, numbers per queue group

Role scoping

A manager's ledger is a join through `team`; floor call-outs are visible to every leader.

```
SELECT i.*
FROM monitoring_instance i
LEFT JOIN team t ON t.team_id = i.team_id
WHERE i.shift_id = $1
  AND (t.manager_id = $2 OR i.person_id IS NULL)
ORDER BY i.created_at DESC;
```

A TL filters on `t.tl_id`; an agent on `i.person_id`.

Strike 3, in one transaction

```
BEGIN;

-- Count the strike; returns 3
INSERT INTO agent_rule_state (shift_id, person_id, rule_id, strikes, fired)
VALUES ($shift, $agent, $rule, 1, true)
ON CONFLICT (shift_id, person_id, rule_id)
DO UPDATE SET strikes = agent_rule_state.strikes + 1, fired = true
RETURNING strikes;

-- Reuse the open incident if there is one; otherwise take a number and open
it
SELECT incident_id FROM incident_log
WHERE person_id = $agent AND rule_id = $rule AND status <> 'Closed';

INSERT INTO incident_counter (year, last_value) VALUES (2026, 1)
ON CONFLICT (year) DO UPDATE SET last_value = incident_counter.last_value + 1
RETURNING last_value;

INSERT INTO incident_log (incident_number, shift_id, source, person_id,
team_id, rule_id)
VALUES ('INC-2026-' || lpad($n::text, 4, '0'), $shift, $source, $agent,
$team, $rule)
RETURNING incident_id;

-- The call-out itself
INSERT INTO monitoring_instance (shift_id, source, rule_id, rule_version,
person_id, team_id,
  metric_value, threshold_value, condition_text, severity, occurrence_count,
notification_level, incident_id)
VALUES ($shift, $source, $rule, $version, $agent, $team,
  $value, $threshold, $condition, 'esc', 3, 'ops', $incident_id);

COMMIT;
```

If two strikes race, `one_open_incident` makes the second insert fail and the transaction retries, finding the incident the first one opened.

Changes from the RTM Data Tables file

The six tables and the flow Event → Rule → Instance → Route → Notification → Incident are kept. These are the places this design departs from the Word file, each to match how the console works.

Word file	This design	Why
One `MonitoringInstance` per agent and rule, updated as `OccurrenceCount` grows	One row per breach; `occurrence_count` is that breach's strike	The ledger, per-call-out acks and comments, and dashboard counts all work per call-out
`IncidentLog.InstanceID` links one instance	`monitoring_instance.incident_id` links many; the incident also holds `person_id` and `rule_id`	An incident gathers strikes 3 and later, and the one-open-incident index needs agent and rule
`RouteMap` as JSON	`notification_route_step` rows, with stage, channels, an open-ended last step and `opens_incident`	The database checks every step; occurrence 4 and later stay at Ops
Threshold, severity and route on `RulesMaster`	On `rule_version`, with `current_version` on the rule	Edits keep history, and each call-out points at the version that fired it
`RuleScope`: agent, team, sub-LOB, LOB, system	`agent` or `floor`	The engine evaluates only those two levels
`Recipients` array with one `DeliveryStatus`	One row per recipient per channel, with `action`	Per-person delivery and what each person did
`SourceEventID` foreign key	Optional column, no foreign key	Duration rules fire from a timer, and old event partitions are dropped

Word file	This design	Why
LOB, sub-LOB and team copied onto each record	`team_id`, with LOB and sub-LOB on `team`	One place to correct, joined when needed
Severity such as "Medium"	`warn`, `crit`, and `esc` at Ops	Matches the engine and the UI
Recipient at occurrence 3 is the Manager	TL, Manager, Admin and Senior Leader	Matches who sees the Ops toast today

Added, because the console needs them: genesys_region, genesys_org, queue, queue_member, `person`, `team`, `shift`, `agent_rule_state`, `push_subscription`, `status_mapping`, `incident_counter` and `audit_event`.

Open decisions

Each of these changes a table, so settle them before the first migration. Numbers refer to Section 10 of `docs/data-architecture.md`.

- [ ] **What a shift is** (decision 7): start and end times, overnight shifts, and whether strikes reset at rollover. Shapes `shift` and `agent_rule_state`.
- [ ] **Team- and LOB-level rules** and per-queue-group numbers (decision 6): needed before `rule_scope` can take the Word file's other values.
- [ ] **Where the org structure comes from** (decision 3): fills `person` and `team`, including TL and manager.
- [ ] **Retention** (decision 9): how many monthly partitions of `source_event_stream` and `notification_log` to keep.
- [ ] **Incidents across shifts:** today an open incident is reused by later strikes on any day. Confirm, or add `shift_id` to the one-open-incident index.
- [ ] **Orgs per region** (Adrian to supply): fills `genesys_org`, with each org's watched view.
- [ ] **One person per org** (Adrian to confirm): decides whether `genesys_account` is needed.
- [ ] **Teams or queues:** today `queue.team_id` points each queue at a team of its own name. Once an org structure source exists, decide whether `team` stays separate from `queue`.